

Trabajo Final – Cliente de Correo Electrónico

Cátedra: Estructuras de Datos en Python

UNaB – Comisión 2

Integrantes: María Lara Zobaji y Mauricio Ramírez

1. Resumen General

El sistema desarrollado modela un cliente de correo electrónico completo utilizando técnicas de programación orientada a objetos y estructuras de datos avanzadas. El objetivo fue representar un ecosistema realista de comunicación entre usuarios, incorporando bandejas, filtros automáticos, prioridades y una red distribuida de servidores.

*Este documento presenta el **UML final** del sistema, basado en los módulos implementados en el repositorio, y describe cada clase, sus relaciones, responsabilidades y estructuras de datos utilizadas. También se justifica la evolución del diseño desde las primeras entregas hasta la integración final.*

2. Vista General del Modelo

El diseño final del proyecto se organiza en cuatro capas:

1. **Modelos (Dominio):** Usuario, Mensaje, Carpeta, ServidorCorreo.
2. **Estructuras de Datos:** Árbol de carpetas, cola de prioridad, grafo de servidores.
3. **Servicios (Lógica):** Gestores para mensajes, carpetas, filtros y red.
4. **Interfaz:** CLI para interacción del usuario.

El sistema sigue un enfoque modular, donde cada componente cumple una responsabilidad clara y se relaciona con otros de manera controlada.

3. Descripción de Clases

3.1 Clase ServidorCorreo (Clase Principal del Sistema)

Representa un servidor capaz de registrar usuarios, almacenar mensajes y participar en la red distribuida.

Atributos:

- `_id` (string): Identificador del servidor.

- `_nombre` (string): Nombre descriptivo del servidor.
- `_usuarios` (dict): Usuarios registrados en este servidor.
- `_conexiones` (list): Otros servidores conectados.
- `_cola_mensajes` (ColaPrioridad): Mensajes pendientes por procesar.

Métodos principales:

- `registrar_usuario()`: Añade un nuevo usuario al servidor.
- `enviar_mensaje_local()`: Maneja el envío dentro del mismo servidor.
- `enviar_mensaje_remoto()`: Envía mensajes hacia otro servidor de la red.
- `recibir_mensaje()`: Procesa la llegada de un mensaje.
- `conectar_servidor()`: Crea una conexión dentro del grafo distribuido.

Relaciones UML:

- Asociación con Usuario (1 servidor → N usuarios)
 - Composición con ColaPrioridad
 - Agregación dentro del grafo de servidores
-

3.2 Clase Usuario

Representa a la persona que utiliza el sistema: posee mensajes, carpetas y filtros.

Atributos (privados):

- `_nombre` (string)
- `_email` (string)
- `_bandeja_entrada` (Carpetas)
- `_bandeja_salida` (Carpetas)
- `_carpetas` (dict)
- `_filtros` (list)

Métodos principales:

- `enviar_mensaje()`
- `recibir_mensaje()`
- `crear_carpetas()`

- listar_mensajes()
- marcar_como_leido()
- asignar_etiqueta()

Relaciones UML:

- Agregación con Carpeta
 - Asociación con Mensaje
 - Asociación con Filtro
-

3.3 Clase Carpeta

Estructura fundamental basada en un **árbol general**, permitiendo jerarquías ilimitadas.

Atributos:

- _nombre (string)
- _mensajes (list<Mensaje>)
- _subcarpetas (list<Carpeta>)
- _carpeta_padre (Carpeta)

Métodos:

- agregar_mensaje()
- mover_mensaje()
- crear_subcarpeta()
- buscar_mensaje_recursoivo()
- listar_subcarpetas_recursoivo()

Relaciones UML:

- Composición con Mensaje
 - Relación recursiva (árbol) Carpeta → Carpeta
-

3.4 Clase Mensaje

Modelo del correo electrónico.

Atributos:

- _id (string)
- _remitente (string)
- _destinatario (string)
- _asunto (string)
- _cuerpo (string)
- _fecha (datetime)
- _prioridad (int)
- _etiquetas (list)

Métodos:

- marcar_como_leido()
- cambiar_prioridad()
- agregar_etiqueta()

Relaciones UML:

- Asociación con Etiqueta
 - Asociado a Usuario y Carpeta
-

3.5 Clase Etiqueta

Permite clasificar mensajes y aplicar filtros automáticos.

Atributos:

- _nombre (string)
- _activo (boolean)

Métodos:

- agregar_etiqueta()
- eliminar_etiqueta()
- listar_x_etiqueta()

Relación principal:

- Composición dentro de Mensaje

3.6 Estructuras de Datos del Sistema

a) ArbolCarpetas

Implementa la lógica interna del sistema de carpetas.

- Inserción: $O(1)$
- Búsqueda recursiva: $O(n)$
- Recorrido completo: $O(n)$

b) ColaPrioridad

Gestiona mensajes urgentes.

- Encolar/Desencolar: $O(\log n)$

c) GrafoServidores

Modela la red distribuida.

- BFS/DFS: $O(V + E)$

Estas estructuras son representadas en UML como objetos asociados a las clases principales.

3.7 Clases de Servicios (Capa Lógica)

Para mantener la separación de responsabilidades, la lógica está distribuida en gestores.

GestorMensajes

- enviar_mensaje()
- recibir_mensajes()
- ordenar_mensajes()

GestorCarpetas

- crear_carpeta()
- buscar_carpeta()
- mover_mensaje()

GestorFiltros

- agregar_filtro()

- aplicar_filtros()

GestorRed

- enviar_mensaje_red()
 - encontrar_mejor_ruta()
-

4. Relaciones UML Principales (Descripción Conceptual)

Composición

- Carpeta → Mensaje
- Usuario → Bandeja de entrada / salida

Agregación

- Usuario → Filtros
- ServidorCorreo → Usuarios

Asociación

- Mensaje ↔ Usuario
- ServidorCorreo ↔ GrafoServidores
- Servicios ↔ Modelos

Relación Recursiva

- Carpeta ↔ Carpeta (árbol)
-

5. Justificación del Diseño

El diseño combina OOP con estructuras de datos adecuadas para cada problema:

- Árbol general: modela la jerarquía de carpetas tal como en un cliente de correo real.
- Cola de prioridad: permite manejo eficiente de mensajes urgentes.
- Grafo: representa conexiones entre servidores y soporta BFS/DFS.
- Encapsulamiento: cada clase tiene atributos privados, accesibles solo mediante métodos.
- Modularidad: separar servicios, modelos y estructuras mejora mantenibilidad.

El modelo evolucionó durante las entregas y esta versión final refleja una arquitectura coherente, escalable y alineada con los requisitos del trabajo.

6. Conclusión

El UML final documenta un cliente de correo que combina estructuras avanzadas y conceptos de orientación a objetos, logrando una simulación completa del proceso de envío, organización y ruteo de mensajes. La división en módulos y servicios facilita su mantenimiento y amplía sus posibilidades de extensión futura.